

Understanding HTTP

Backend Architecture and HTTP Fundamentals

1. What Is Backend Architecture?

Backend architecture is one of the most crucial parts of full-stack development, yet it is often misunderstood. When people first hear the word *backend*, they usually think of vague concepts such as servers, databases, the cloud, APIs, Kubernetes, or microservices. However, none of these terms actually explain what a backend is or why it exists.

To understand backend architecture, it is better to strip away the tools and focus on the problem the backend is trying to solve.

The Front End Cannot Be Trusted

At its core, a backend exists because the front end cannot be trusted. Anything that runs on a user's device is fundamentally under the user's control.

If an application runs in a browser, the user can open developer tools, modify JavaScript, change network requests, or completely fake inputs. If it runs on a mobile device, the user can intercept traffic, tamper with requests, or reverse-engineer the application.

The front end is therefore not a secure environment, and it never will be.

Because of this, any application that needs rules, business logic, real data, or security cannot rely on the front end alone.

The Backend as the System of Authority

The backend becomes the system of authority. It is the place where decisions are made about what is allowed to happen and what is not.

The front end can ask for something, but the backend decides whether that request is valid.

At a high level, the way a backend works is simple:

1. It waits for incoming requests.
2. It processes those requests.
3. It sends back a response.

A request usually comes from a client such as a browser or mobile application. The backend reads the request, validates it, checks permissions, applies business logic, and interacts with storage or other services when necessary. Finally, it returns a response that tells the client what happened.

The request-response loop is the fundamental unit of backend execution. Every backend system, regardless of how large or complex it becomes, is built around this basic loop.

Backend and Shared State

Another critical reason backends are needed is shared state.

In any real application, multiple users interact with the same system. They need to see consistent data. If one user creates or updates something, another user should be able to see that change.

This is why the backend is often considered the **source of truth**.

The backend is the final authority on:

- What the current state of the system is.
- What transactions are allowed.
- What changes can occur.
- Who is authorized to make those changes?

The front end is simply a way for users to interact with that system.

Once you understand backend architecture in terms of responsibilities rather than tools and technologies, it becomes much easier to reason about.

2. HTTP: How the Front End Communicates With the Backend

Once we understand what a backend is and why it exists, the next question is obvious:

How does the front end actually communicate with the backend?

The front end does not magically call the backend. There has to be a shared language—a set of rules that both sides agree on.

That shared language is **HTTP**, or **Hypertext Transfer Protocol**.

HTTP Is a Communication Convention

The name "Hypertext Transfer Protocol" can make HTTP sound more complicated than it really is.

You do not need to think about hypertext or protocols in an abstract sense. HTTP is simply a convention—an agreement that defines how a client sends a message to a server and how the server responds.

At a fundamental level, HTTP is not about websites. It is not specifically about browsers, and it is not even about HTML.

HTTP is about **structured message exchange between two machines**.

One side asks for something, and the other side responds.

Everything else is built on top of that idea.

The Basic HTTP Interaction

At a simple level, we have:



- A **client**, such as a browser or mobile application.
- A **backend**, running on a server.

Whenever a user performs an action such as logging in, creating an account, retrieving data, or saving information, the client sends an HTTP request to the server.

The server then:

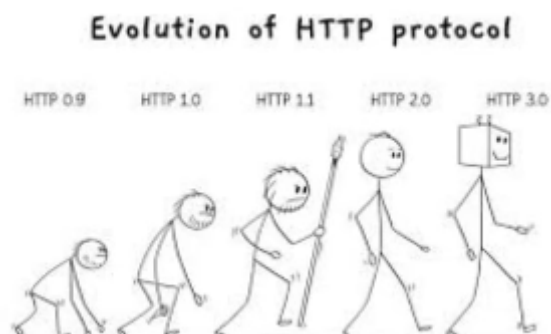
1. Reads the request.
2. Interprets it.
3. Validates it.
4. Performs the required operation.
5. Sends an HTTP response back.

The response tells the client whether the operation succeeded or failed and may contain additional information.

3. Evolution of HTTP

HTTP did not appear fully formed. It evolved over time in response to real problems encountered by the web.

Understanding those problems helps explain why HTTP looks the way it does today.



Early HTTP

The original use case for HTTP was simple document retrieval.

A client would ask a server for a document, usually an HTML file, and the server would send it back.

There were:

- No APIs.
- No JSON.
- No complex interactions.
- No sophisticated metadata.
- No modern application behavior.

The earliest versions of HTTP were therefore extremely minimal.

The client sent a request, the server returned raw content, and the connection closed.

HTTP/1.0

As the web grew, servers needed a way to communicate more information than simply returning a document.

They needed to communicate things such as:

- Whether the request succeeded.
- What type of data was being returned.
- How the client should interpret the response.

HTTP/1.0 introduced meaningful support for **headers and status codes**.

Instead of guessing what happened, the client could read an explicit status code.

Instead of assuming everything was HTML, the client could read the content type.

This made the web significantly more reliable because clients and servers could evolve independently as long as they respected the shared message structure.

HTTP/1.1

HTTP/1.0 still had a major inefficiency: every request required a new connection.

HTTP/1.1 addressed this problem by introducing **persistent connections**.

Instead of opening and closing a connection for every request, the client and server could reuse the same connection for multiple requests.

HTTP/1.1 also introduced or standardized improvements such as:

- More precise caching controls.
- Chunked transfer encoding.
- Stricter message-formatting rules.
- Better connection management.

HTTP/2 and HTTP/3

Later versions such as HTTP/2 and HTTP/3 focus heavily on performance and transport efficiency.

The fundamental mental model does not change.

You still have:

Requests → Responses

What changes is how efficiently those messages are transported across the network.

This evolution demonstrates an important lesson in backend architecture:

Protocols evolve because real systems encounter real limitations.

HTTP survived because it adapted without abandoning its core contract.

4. Structure of an HTTP Message

Backend systems ultimately deal with bytes arriving over the network. Those bytes must be interpreted correctly.

Every framework, server, and reverse proxy is ultimately parsing HTTP messages and turning them into something application code can work with.

There are two primary types of HTTP messages:

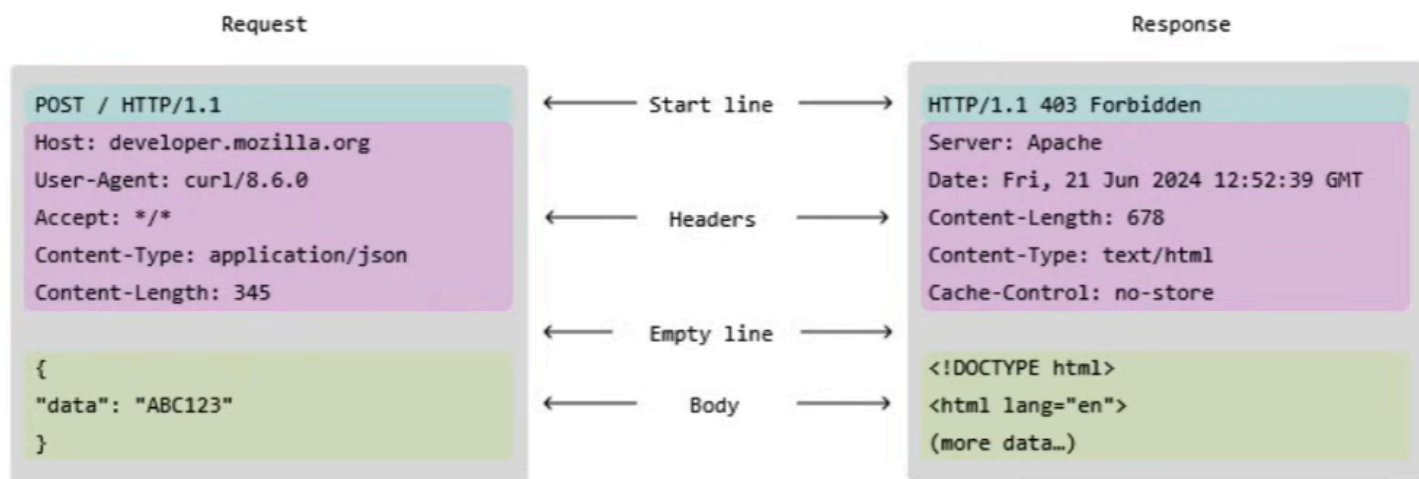
- **Requests**
- **Responses**

Requests are sent by clients to describe what they want to do.

Responses are sent by servers to describe what happened.

Despite their differences, both follow a common structure.

HTTP Message Structure



An HTTP message generally contains:

1. A start line.
2. Headers.
3. An empty line.
4. An optional body.

The structure exists so that machines can reliably parse messages without ambiguity.

The Start Line

Every HTTP request begins with a start line.

The request start line tells the server: **POST / HTTP/1.1**

1. What action the client wants to perform. **POST**
2. What resource is being targeted. **HTTP**
3. Which version of HTTP is being used. **1.1**

The response start line works differently. Instead of describing intent, it describes the outcome.

It tells the client: **HTTP/1.1 403 Forbidden**

- Which HTTP version is being used. **HTTP/1.1**
- Whether the request succeeded or failed. **403**
- What kind of result occurred **Forbidden**

HTTP Headers

After the start line come the headers.

Headers are collections of metadata fields that answer specific questions about the message.

For example:

- **Content-Type** tells the receiver how to interpret the body.
- **Authorization** communicates authentication information.
- **Content-Length** indicates how much data should be expected.

Headers provide context surrounding the actual message body.

The Empty Line

The empty line separating the headers from the body is extremely important.

It acts as a boundary between **metadata** and **content**.

This distinction allows the receiver to understand where the headers end and where the actual payload begins.

The Body

The body contains the actual data being sent.

It may contain:

- JSON.
- HTML.
- Text.
- Binary data.
- Form data.
- Files.
- Other representations.

Not every HTTP message has a body.

For example, many **GET** requests do not contain a request body, and some responses do not contain one either.

5. Why HTTP Headers Exist

Once we understand that HTTP messages contain a start line, headers, and an optional body, an obvious question arises:

Why do we need headers if the body already carries data?

The answer is that headers separate **context from content**.

The body carries the actual payload.

Headers describe that payload and the conditions under which the message should be handled.

Headers Separate Context From Content

Without headers, every message would require custom parsing rules.

A server would not know:

- What format the body uses.
- Whether the client is authenticated.
- How large the message is.
- Whether it can be cached.
- How it should be processed.

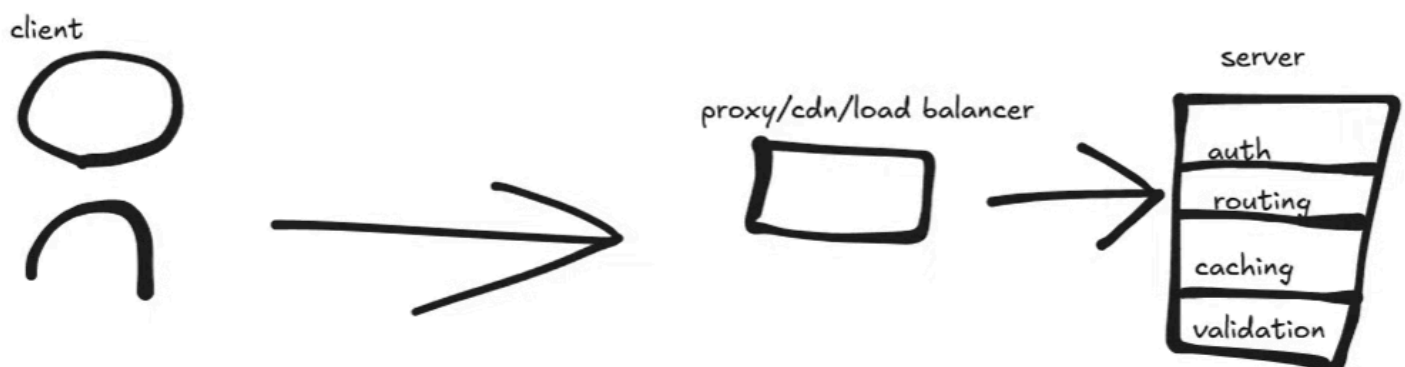
Every application would have to reinvent this logic.

Headers solve this by providing a standardized location for metadata.

This allows the body to remain focused on data while surrounding context remains in a predictable, machine-readable format.

Headers and Intermediaries

HTTP traffic rarely travels directly from a client to a backend.



Requests and responses often pass through:

- Proxies.
- Load balancers.

- CDNs.
- Gateways.
- Caches.

These systems need to make decisions without understanding application-specific data.

Headers provide the information needed to:

- Route requests.
- Cache responses.
- Compress data.
- Reject requests.
- Manage connections.

This allows intermediaries to operate without needing to understand the application's internal business logic.

Headers Make HTTP Extensible

Headers also make HTTP extensible.

New headers can be introduced over time without fundamentally changing the message structure.

Older clients can ignore headers they do not understand, while newer clients and servers can use them.

This design is one reason HTTP has remained useful for decades.

6. Categories of HTTP Headers

HTTP headers are not just a random collection of fields. They fall into categories and those categories exist because different headers are meant to influence different stages of request and response handling. At a high level, headers can be grouped based on what they describe. Some headers describe the message itself, some describe the client, some describe the server's response, and some describe how intermediaries should treat the message. These distinctions matter because different systems in the HTTP pipeline care about different types of headers. A browser might care about one category, a CDN might care about another, your backend application might care about a third. Understanding header types also helps you design cleaner APIs and you stop stuffing information into the body that doesn't belong there.

They can be conceptually grouped according to what they describe and what part of the HTTP pipeline uses them.

A traditional conceptual grouping contains four categories:

1. General headers.
2. Request headers.
3. Response headers.
4. Entity headers.

These categories are primarily useful for understanding responsibilities rather than memorizing terminology.

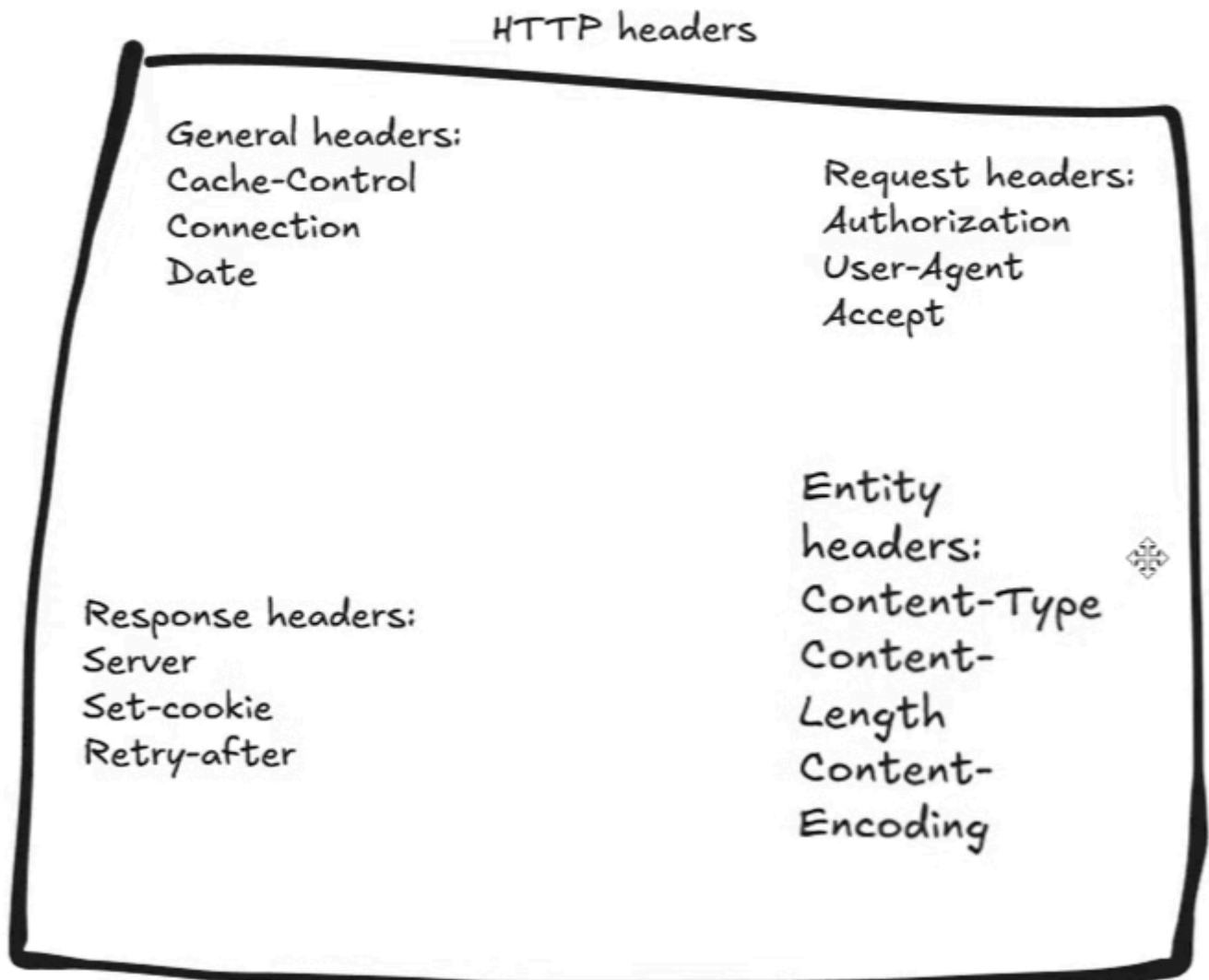
General Headers

General headers apply to both requests and responses.

They describe how the message should be handled overall.

They can influence things such as:

- Caching.
- Connection handling.
- Timing.
- General protocol behavior.



Request Headers

Request headers describe the client and what it wants.

They can communicate:

- Who is making the request.
- What formats the client can accept.
- Authentication information.
- Expectations about how the server should behave.

Authentication and content negotiation often rely heavily on request headers.

Response Headers

Response headers describe what the server did and how the client should react.

They can communicate:

- Caching instructions.
- Resource information.
- Security policies.
- Limits.
- Expectations for future requests.

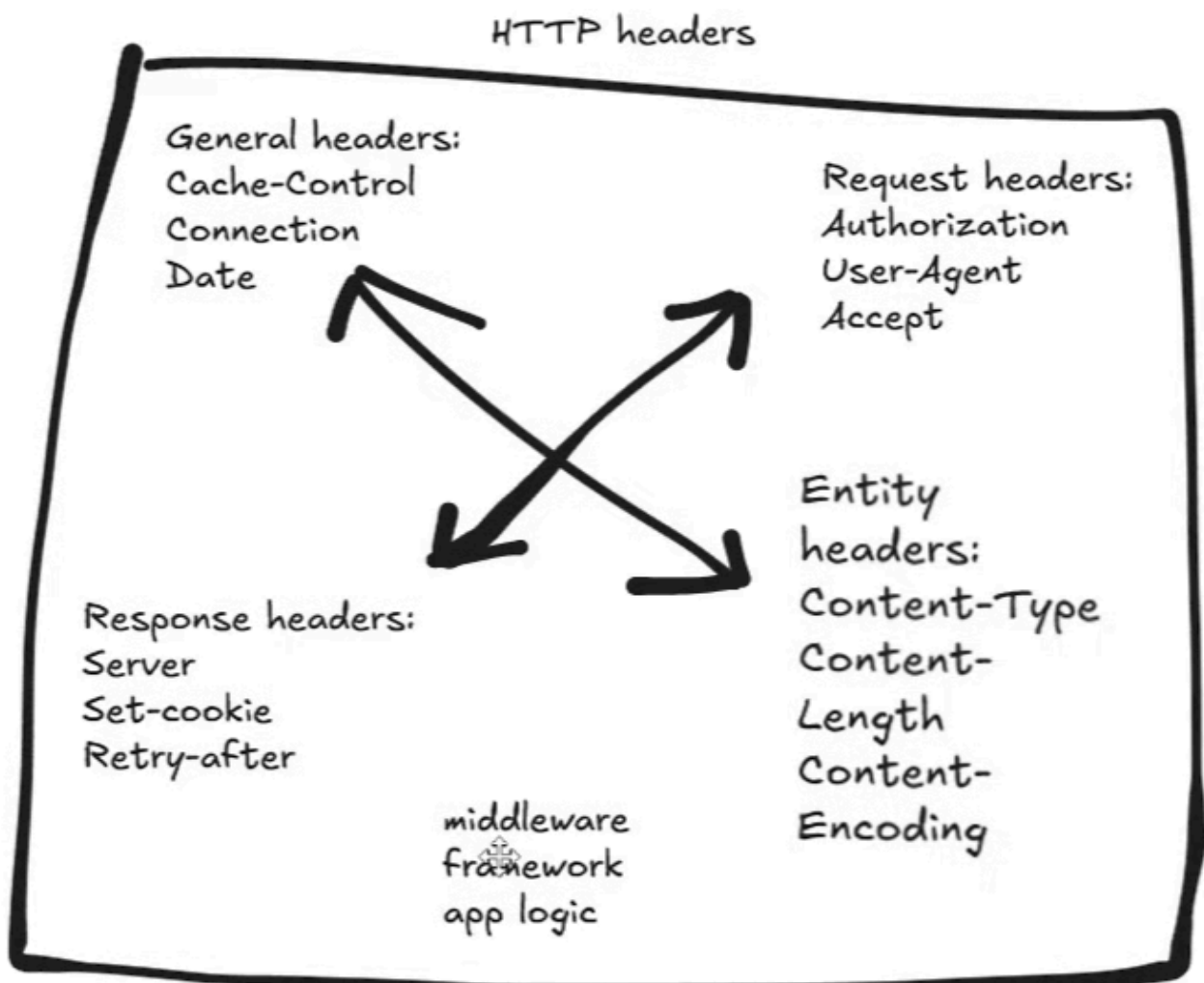
Entity or Representation Headers

Entity headers, using older HTTP terminology, describe the message body or representation.

They can communicate:

- The type of data.
- The size of the data.
- Encoding information.
- Other characteristics of the representation.

The exact classification of individual headers can vary, so the goal is understanding their responsibilities rather than memorizing rigid categories.



Headers are often processed before your code even runs. Middleware reads them, frameworks interpret them, and your application receives a filtered view of the request. By the time your controller runs, many headerdriven decisions have already been made.

7. HTTP Methods

HTTP methods describe the action a client wants to perform.

One of the biggest beginner mistakes is treating HTTP methods as arbitrary labels.

Methods such as **GET**, **POST**, **PUT**, and **DELETE** have specific semantics.

These semantics influence:

- Caching.
- Security.
- Retries.
- API design.
- Infrastructure behavior.
- Failure handling.

Using the correct method helps APIs remain predictable.

Common HTTP Methods

GET

GET is used to retrieve data.

A **GET** request should not intentionally modify server state.

POST

POST is commonly used when the client wants to submit data or trigger an operation that causes a server-side change.

Examples include:

- Creating a resource.
- Submitting a form.
- Starting a process.
- Triggering an action.

PUT

PUT is generally used to replace a resource at a known location.

The client is effectively communicating:

This is the new representation of this resource.

PATCH

PATCH is used for partial updates.

Instead of replacing the entire resource, it modifies a specific portion of it.

DELETE

DELETE is used to remove a resource.

HEAD

HEAD behaves similarly to **GET**, except that the response does not contain the response body.

It can be useful when the client only needs metadata such as:

- Headers.
- Cache information.
- Content length.

OPTIONS

OPTIONS asks the server what communication options are supported.

It becomes particularly important in browser-based cross-origin communication and CORS.

8. Idempotency and Reliable Retries

One of the most important concepts behind HTTP methods is **idempotency**.

An operation is idempotent if performing it multiple times yields the same result as performing it once. Just the key idea is repeatability without side effects. This matters because networks are unreliable. Requests can get dropped, responses time out, and clients retry, proxies retry, etc. When retries happen, the server may receive the same request more than once. And if an operation is idempotent, retries are safe. If it's not, retries can cause duplicated data, inconsistent states, and very hard to debug issues.

A client might send a request and never receive the response. It then has to decide whether to retry. This retry may send the same exact request again, but the server doesn't know it's a retry unless we design for it. If the operation is idempotent, repeating the request can be safe.

Idempotent

GET
PUT
DELETE
HEAD
OPTIONS

Non-Idempotent

POST

Idempotent Operations

Commonly idempotent HTTP methods include:

- **GET**

- PUT
- DELETE

For example, replacing a resource with the same representation multiple times results in the same final state.

Similarly, deleting an already deleted resource does not normally produce an additional state change.

Non-Idempotent Operations

POST is generally not idempotent.

Sending the same POST request multiple times can create multiple resources or trigger the same operation multiple times. If we send the same PUT request, it will repeatedly yield the same version of a resource

This creates a challenge for systems that need reliable retries.

Idempotency Keys

In real systems, we often add safeguards to make non-idempotent operations safer. Idempotent keys allow the server to recognize repeated requests and ignore duplicates. This is common in payment processing, order creation, and mission critical workflows. Idempotent is simply about trust. It allows a system to retry safely without coordination. It's one of the quiet assumptions that makes distributed systems work at scale.

An **idempotency key** allows the server to recognize repeated attempts to perform the same operation.

This is particularly useful for:

- Payment processing.
- Order creation.
- Financial transactions.
- Mission-critical workflows.

Idempotency is ultimately about making systems behave predictably under failure.

9. CORS and Cross-Origin Requests

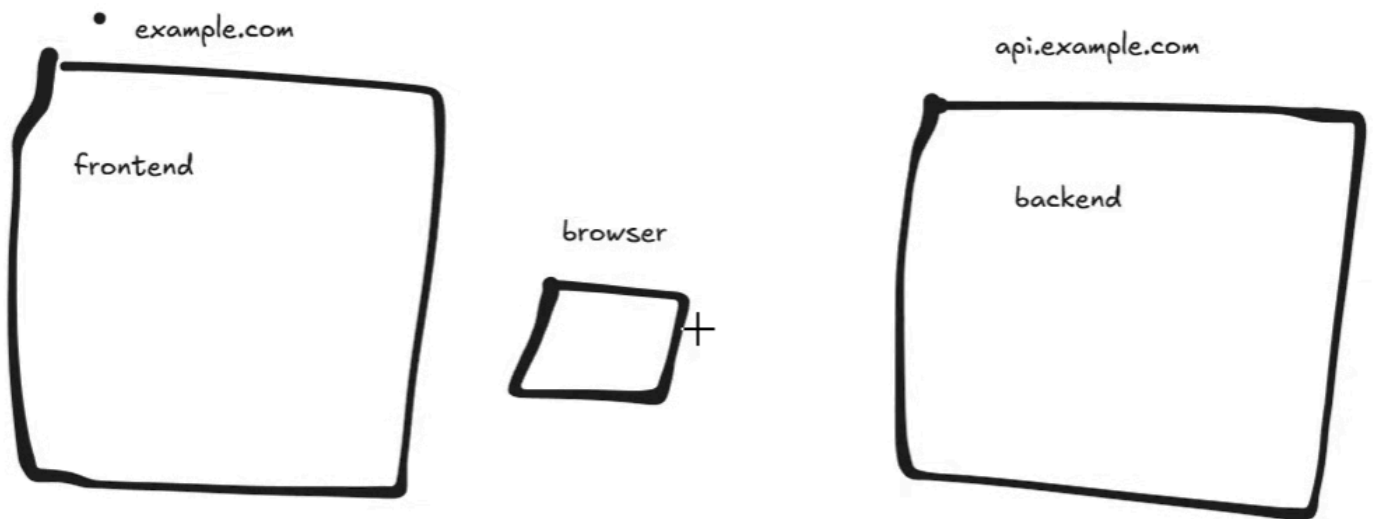
Modern web applications often have frontends, backends, APIs, and third-party services hosted on different origins.

This creates an important security problem:

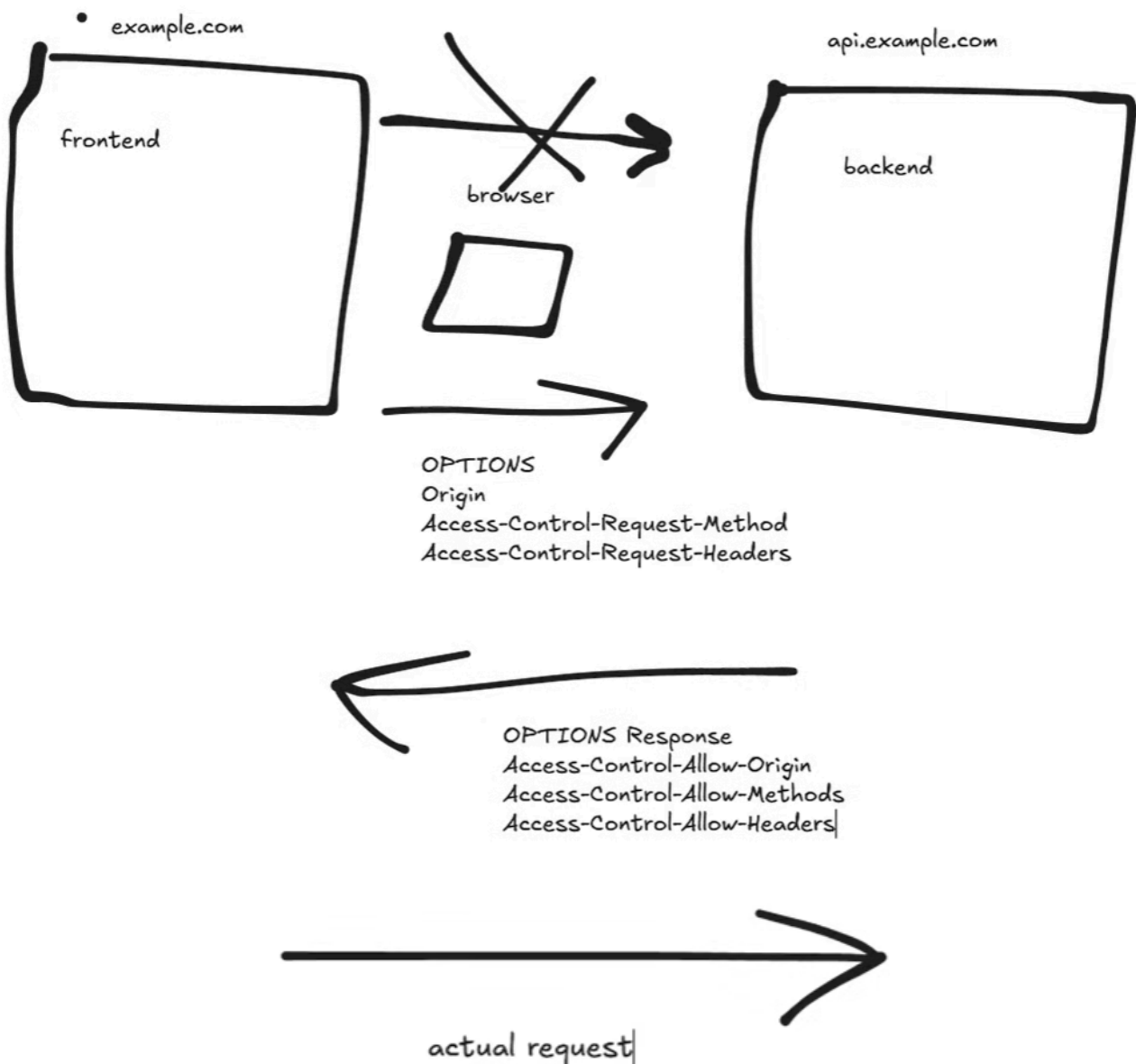
How does a browser know whether it is safe to allow one origin to communicate with another?

This is where **Cross-Origin Resource Sharing (CORS)** comes into play.

CORS is primarily a browser security mechanism. The server participates in the process, but the browser ultimately enforces the policy. And at the center of that workflow is the options method. Options are often misunderstood or ignored because they don't carry business data. But without it, modern web apps wouldn't function securely. Options exist. So a client can ask the server what is allowed to do before doing it. CORS is fundamentally about trust boundaries. Browsers assume that different origins should not automatically trust each other. Before sending certain requests, the browser performs a preflight check to see whether the server explicitly allows the interaction or not.



If we take a front end and a back end, even though these look related, the browser treats them as different origins. Different origins means different security context. The browser is not a passive messenger. It actively enforces these security rules. And this enforcement happens before your backend code runs.



For certain requests, the browser will not immediately send the actual request because it may need permission first. This pre-flight request asks the server three questions. Who's making the request? What method will be

used? And what headers will be sent? No data is modified during this setup. The server responds with explicit permissions. It doesn't say maybe. It's a definite yes or no. This response determines whether the browser proceeds. If the response allows the request, the browser proceeds. If not, the browser blocks it entirely and the backend may never even see the real request. Only after a successful pre-flight does the browser send the real request. This is why you'll sometimes see options requests and logs before a post or a put request.

Simple Request = no preflight
Complex Request = preflight

In essence, if a request is labeled as simple, no pre-flight is required, it'll skip it. Others require it. CORS isn't something that you can just fix with code alone. It's a contract between the browser and server. Options exist to allow that contract to be negotiated safely.

From a backend perspective, CORS is a reminder that not all methods or HTTP behavior is controlled by your server. Clients, especially browsers, have their own rules.

Origins and Trust

Browsers do not automatically trust different origins.

A frontend and backend may be related parts of the same application from a developer's perspective, but the browser can still treat them as different security contexts.

For certain cross-origin requests, the browser performs a **preflight request** before sending the actual request.

The OPTIONS Preflight Request

The preflight request commonly uses the **OPTIONS** method.

It allows the browser to ask the server what is permitted.

The browser can communicate information such as:

- The requesting origin.
- The HTTP method it intends to use.
- The headers it intends to send.

The server then responds with the appropriate CORS permissions.

If the browser determines that the request is allowed, it proceeds with the actual request.

If the browser determines that it is not allowed, it blocks the request.

This is why backend logs can sometimes show an **OPTIONS** request before a **POST**, **PUT**, or other request.

Not every cross-origin request requires a preflight. Certain requests qualify as "simple" and can skip this step.

CORS demonstrates an important backend principle: not all HTTP behavior is controlled solely by the server. Browsers have their own security rules that affect whether a request reaches the backend.

10. HTTP Status Codes

Once a server receives a request and processes it, it needs a way to communicate that result back to the client.



That communication does not start with the response body. Rather, it starts with a status code. Status codes aren't just numbers. They're signals. They tell the client, intermediaries, and even automated systems how to interpret the response without reading the body at all.

A well-designed backend uses status codes intentionally. A poorly designed backend treats everything as a 200 and stuffs error information into the body uh that will break tooling, caching, retries, and observability. Status codes exist so that systems react correctly. A browser behaves differently for a 301 than a 401. A load balancer behaves differently for a 503 than a 404. And clients can retry some errors and must stop on others. Every response starts with a status code.

Status Code Categories

Status codes are grouped according to their first digit.

1xx — Informational

These indicate that a request has been received and that additional processing is taking place.

They are relatively uncommon in typical application development.

2xx — Success

These indicate successful processing.

Common examples include:

- **200 OK** — The request succeeded and data is returned.
- **201 Created** — A resource was successfully created.
- **204 No Content** — The request succeeded without a response body or content.

3xx — Redirection

These tell the client that it needs to take another action or retrieve the resource elsewhere.

Redirections are not necessarily errors but treated as instructions.

Browsers, crawlers, and caches use them extensively.

4xx — Client Errors

These indicate that there was a problem with the request or the client's ability to perform the operation.

Examples include:

- Invalid input.
- Missing authentication.
- Insufficient permissions.
- Requesting a resource that does not exist.

Repeating the exact same request will often not solve the problem.

5xx — Server Errors

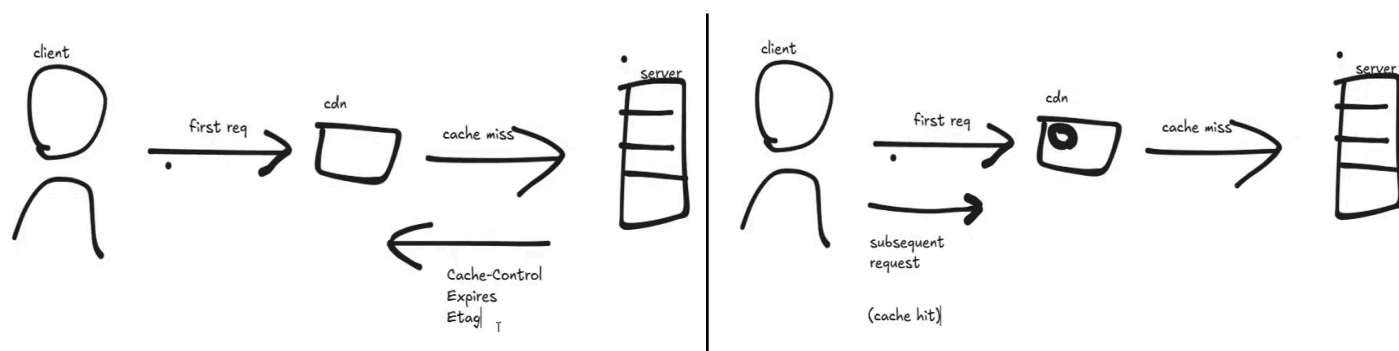
These indicate that the server failed while processing an otherwise valid request.

These errors can be particularly important for monitoring and retry behavior.

11. HTTP Caching

One of the biggest reasons HTTP has been able to scale globally is caching. Without caching, every request would have to hit the origin server. That simply doesn't work at internet scale. Caching allows responses to be reused instead of recomputing or refetching the same data over and over again. Systems can serve previously stored responses as long as they're still valid.

How HTTP Caching Works



When a resource is requested for the first time, the cache may not have a stored response.

The request therefore travels to the server.

The server returns a response containing metadata that tells the cache how the response should be treated.

The cache stores the response and its associated metadata.

When another request for the same resource arrives, the cache can determine whether the stored response is still valid.

If it is, the cache can respond directly without contacting the backend.

This provides two major benefits:

- Faster responses for clients.
- Reduced load on backend infrastructure.

Caching behavior is heavily influenced by HTTP headers.

12. Content Negotiation

Different clients may want different representations of the same resource.

For example:

- A browser may want HTML.
- A mobile application may want JSON.
- A script may want XML or plain text.

This is where **HTTP content negotiation** comes in. It exists so the client and server can agree on how data should be represented without changing the underlying resource. This is a very important distinction. Content negotiation is not about changing what the data is. It's about changing how the data is presented.

The Accept Header

The client can use the **Accept** header to communicate which formats it understands or prefers.

For example, a client might indicate that it prefers JSON.

The server compares the client's preferences with the formats it can produce and chooses an appropriate representation.

The server then communicates the actual representation using the **Content-Type** response header.

This approach avoids requiring separate endpoints for every possible representation.

13. HTTP Compression

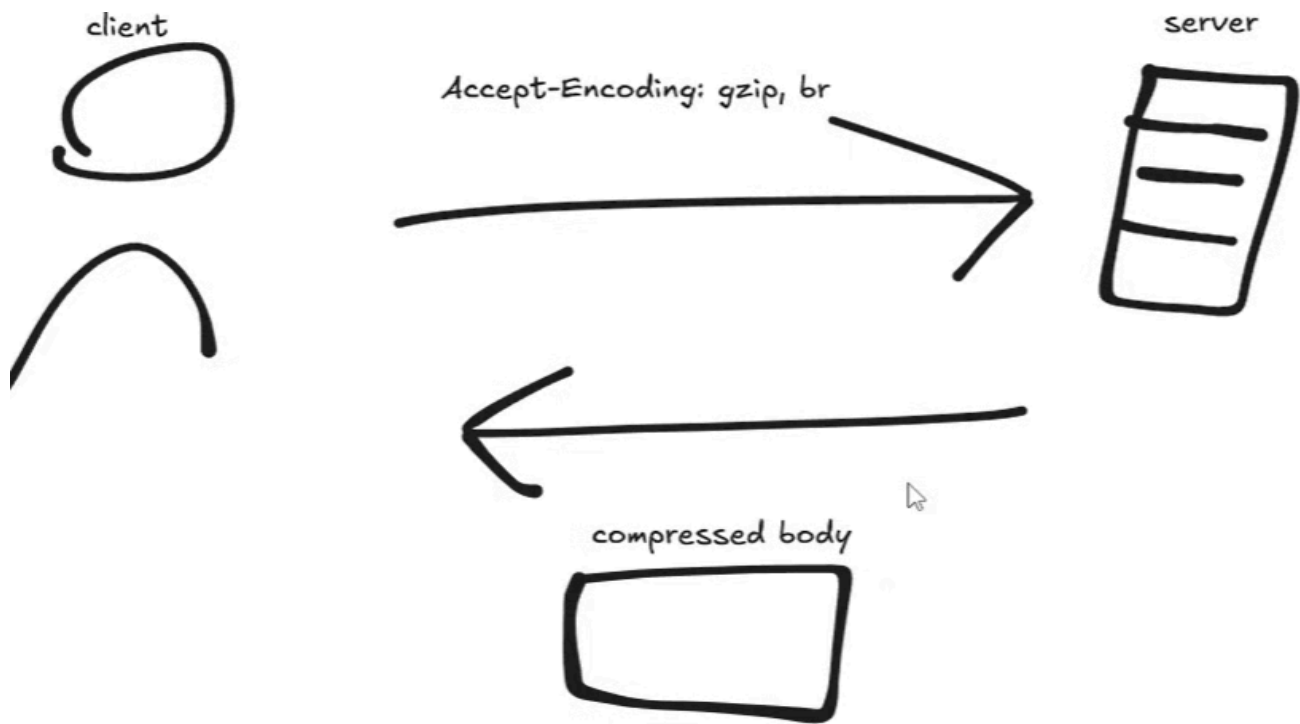
Once a client and server agree on the format of the data, the next concern is size. Transferring data over a network costs time, bandwidth, and money. Compression exists to reduce the amount of data sent over the wire without changing the actual content. Compression doesn't change what the data means. It simply changes how efficiently it's transmitted.

How Compression Works

The client can communicate which compression algorithms it supports.

The server chooses an appropriate supported encoding and sends the response in compressed form.

The client then decompresses the response.



The application itself does not need to understand the compression process.

It is handled at the protocol and transport level.

Although the savings on a single request may be small, the benefits become significant when multiplied across millions of requests.

14. Persistent Connections

Every HTTP request requires a connection between the client and the server. But that connection is not free. It requires a handshake, resource allocation, and coordination on both sides. If a new connection were to be created for every single request, performance would suffer dramatically. Early versions of HTTP did exactly that. Each request, open a new connection, sent the request, received the response, and then closed the connection.

For a modern application containing many resources, this approach creates substantial overhead.

Non-Persistent Connections

A non-persistent connection follows a simple pattern:

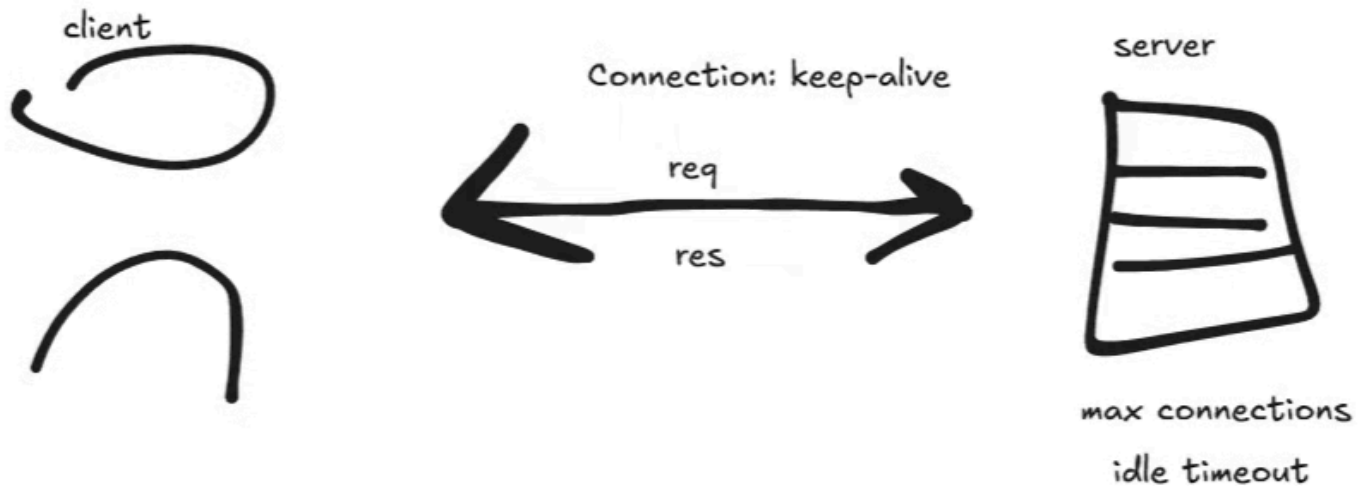
Open connection → Send request → Receive response → Close connection

If a page contains many images, scripts, stylesheets, and API calls, repeatedly creating connections becomes inefficient.

Persistent Connections and Keep-Alive

Persistent connections allow the client and server to reuse an existing connection for multiple requests until it stays open, and this dramatically reduces the overhead.

Persistent Connections and Keep-Alive



We can use a connection header to specify this as keep alive. And this header tells the server the client wants to reuse the connection. And in modern HTTP, persistent connections are the default. But this is where the concept comes from. But the servers don't keep these connections open forever. They enforce limits to protect resources like idle timeouts and max connections limits.

This reduces:

- Connection overhead.
- CPU usage.
- Latency.
- Resource consumption.

Persistent connections are therefore an important part of efficient network communication.

15. Multipart Data

Up to this point, We've talked mostly about HTTP messages where the body is a single clean block of data. That works great for small payloads, but real systems often need to send large files, mixed data types, or data that isn't fully known ahead of time. And this is where multi-art data is chunked, transferred, encoded. They solve different problems, but both exist to make HTTP flexible enough to handle real world data transfer. Multipart data is about structure and it allows a single message body to contain multiple distinct parts each with its own headers and content type. Chunk transfer encoding is about timing. It allows data to be sent progressively without knowing the total size up front. Multiart data is most commonly encountered when handling file uploads, especially when a request includes both files and structured metadata. A single JSON body is not enough when you need to send binary data alongside text fields. Multiart solves this by turning the body into a container that holds multiple smaller bodies, each with its own description.

HTTP bodies do not always contain a single simple block of data.

Real applications often need to send:

- Files.

- Text fields.
- Structured metadata.
- Binary data.

This is where **multipart data** becomes useful.

Multipart Requests

A multipart message allows a single HTTP body to contain multiple distinct parts.

Each part can have its own:

- Headers.
- Content type.
- Content.

The different sections are separated by special **boundaries**.

This allows binary data and text data to coexist within a single HTTP request.

File Uploads

Multipart requests are commonly used for file uploads.

For example, an application might need to submit:

- A user's profile image.
- The user's name.
- Additional metadata.

All of this can be transmitted as different parts of the same HTTP request.

16. Chunked Transfer Encoding

Multipart data and chunked transfer encoding solve different problems. Sometimes the server does not know how large the response will be when it starts sending. Streaming data, live feeds, or dynamically generated content all fall into this category. Instead of buffering the entire response, HTTP allows the server to send data in chunks as it becomes available.

Multipart data is primarily about **structure**.

Chunked transfer encoding is primarily about **timing and streaming**.

Sometimes a server does not know the total size of a response before it starts sending it.

Examples include:

- Streaming data.
- Live feeds.
- Dynamically generated content.

If the server doesn't know the final size, then it can't set the content length, and chunked encoding solves this. You can think of these as multiple chunks. The response body is broken into these chunks. Each sent as it's ready. The client processes them incrementally as they receive them.

Both of these concepts are simply about flexibility and they allow HTTP to handle complex large or streaming data without breaking its core structure.

Sending Data in Chunks

The response body can be broken into multiple chunks.

Each chunk is sent as it becomes available.

The client can process those chunks incrementally rather than waiting for the entire response.

This makes HTTP flexible enough to support large and dynamically generated responses without requiring the server to buffer everything in memory first.

17. SSL, TLS, and HTTPS

So everything we've discussed assumes something dangerous that the data traveling between the client and the server is safe. By default, it shouldn't be.

Plain HTTP does not provide that protection. Anyone sitting between the client and the server can read it, modify it, or impersonate one side of the conversation, and that includes **passwords, tokens, cookies, and personal data**. SSL and TLS exist to solve this problem. And they don't necessarily replace HTTP, but they wrap it. And HTTPS is just simply HTTP over a secure encrypted connection.

The tunnel around the connection is TLS and TLS encrypts data before it leaves the client and decrypts it only when it reaches the server. Everything inside this tunnel is protected. HTTPS simply is just HTTP included with TLS. All of the HTTP semantics remain unchanged. This is a basic overview of the TLS handshake.

TLS

TLS (Transport Layer Security) provides encryption and authentication for network communication.

HTTPS is simply:

HTTP over TLS

TLS does not replace HTTP. Instead, it protects the communication channel through which HTTP messages travel.

The HTTP semantics remain the same.

What Happens During a TLS Handshake?

Before secure communication begins, the client and server perform a TLS handshake.

At a simplified level:

TLS handshake overview



1. The client sends a hello message.
2. The server responds with its own hello.
3. The server provides a certificate.
4. The client verifies the server's identity.
5. The parties establish shared cryptographic keys.
6. Encrypted application data can then be exchanged.

The server certificate helps the client verify that it is communicating with the intended server.

Once the handshake is complete, HTTP requests and responses—including headers, bodies, cookies, and other application data—are transmitted through the encrypted connection.

18. Bringing Everything Together

HTTP should not be viewed as a random checklist of features.

It is a layered system in which multiple mechanisms work together:

- **Methods** communicate intent.
- **Status codes** communicate outcomes.
- **Headers** communicate context and metadata.
- **Bodies** carry content.
- **Caching** reduces unnecessary work.
- **Content negotiation** allows different representations.
- **Compression** reduces transfer size.
- **Persistent connections** reduce connection overhead.
- **Multipart requests** support complex payloads.
- **Chunked transfer** supports progressive transmission.
- **CORS** manages browser-enforced cross-origin access.
- **TLS** protects HTTP communication.

Frameworks and libraries abstract much of this complexity away, but they do not remove it.

When a framework gives you a request object, it is ultimately representing an HTTP request.

When a framework gives you response helpers, they are ultimately constructing an HTTP response.

When a reverse proxy routes traffic, it is interpreting HTTP information.

When a CDN caches a resource, it is using HTTP semantics and metadata.

Understanding these underlying layers gives backend engineers the ability to reason about systems instead of simply memorizing framework features.

19. Final Perspective

Backend architecture becomes much easier to understand when you stop thinking primarily in terms of tools such as databases, APIs, Kubernetes, microservices, and cloud platforms.

The deeper question is:

What responsibility does the backend have?

The backend exists because clients cannot be trusted to enforce the rules of the system. It acts as the authority that validates requests, checks permissions, applies business logic, manages shared state, and determines what changes are allowed.

HTTP provides the communication contract between clients and servers.

Its structure gives us a predictable way to exchange messages. Methods communicate intent. Headers provide context. Bodies carry data. Status codes communicate outcomes. Caching, negotiation, compression, persistent connections, streaming, and TLS extend the protocol to handle the requirements of modern applications.

Once these concepts are understood, backend architecture stops looking like a collection of unrelated technologies.

Instead, it becomes a system of responsibilities and layers designed to solve specific problems.

Frameworks abstract these details away, but they do not eliminate them.

If you understand HTTP deeply, you gain the ability to reason about backend systems, debug production problems, design APIs, understand infrastructure, and make informed architectural trade-offs.

You are no longer simply writing backend code.

You are designing systems.